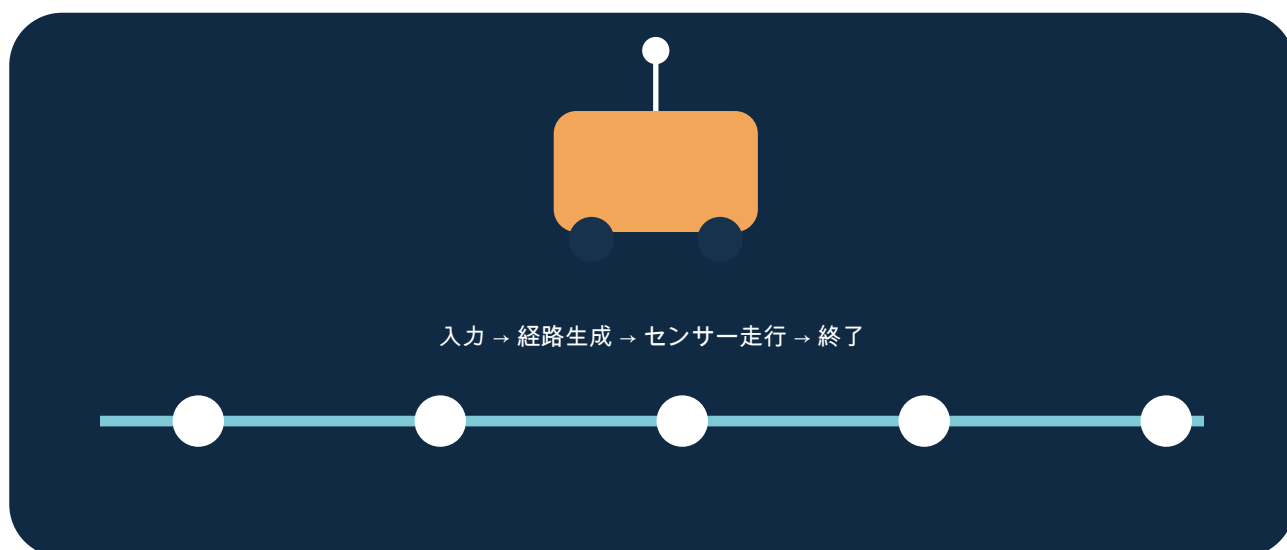


ProgramB

初学者向けコード解説

入力した交差点を順番にたどるラインレースロボット。

コードを初めて見る人が「何が、いつ、なぜ動くのか」をつかむための完全ガイドです。



対象ファイル: ProgramA.ino / ProgramB.ino / ProgramTypes.h

作成日: 2026年8月1日

この資料の読み方

最初に覚えるのは3つだけ

① 入力した地点名を座標にする ② 座標から f/r/l の経路を作る ③ センサーを見ながら命令を順番に実行する

このPDFは、C/C++やArduinoを知らなくても読み進められる順番にしています。最初に全体像をつかみ、その後で変数、関数、具体例、センサー制御の順に細部へ降りていきます。コード中の名前は等幅文字で示します。

目次

章	内容	おすすめの読み方
1	このロボットがすること / 3ファイルの役割	まず読む
2	最低限のプログラム用語	コードが完全に初めてなら読む
3	座標・方角・モード	データの持ち方を理解する
4	起動から終了までの全体フロー	処理の順番をつかむ
5	変数・定数の完全一覧	名前がわからなくなったら戻る
6	関数の完全一覧	各部品の役割を調べる
7	入力と経路生成を詳しく	文字列が経路になる仕組み
8	走行・センサー・旋回を詳しく	ロボットが実際に動く仕組み
9	エラー対策と調整値	安全策とチューニング
10	コードを読む練習 / 用語集	復習に使う

記号について

→ は処理の流れ、true / false は成功・失敗などの二択、*p は「元のデータを書き換えるための住所」を表します。

1. このロボットがすること

シリアルモニタから 159b. のように交差点名を入力すると、ロボットは $1 \rightarrow 5 \rightarrow 9 \rightarrow b$ の順に通る経路を作り、そのラインを追って行きます。最後のセリオドは入力終了印です。

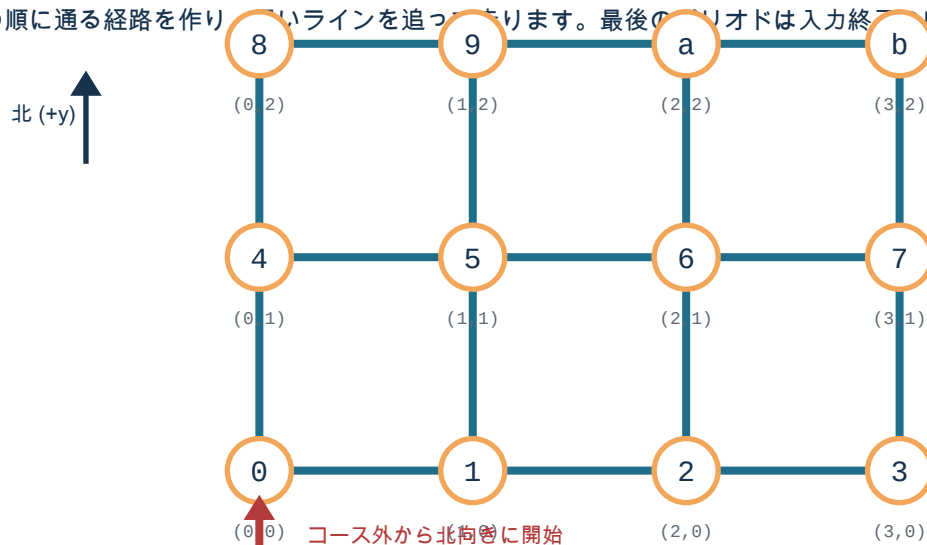


図1 交差点ラベルと座標。右が +x、上が +y。ロボットは交差点0の下から北向きに入ります。

3つのファイルの分担

ファイル	主な役割	たとえるなら
ProgramA.ino	初期化、行き先の入力、地点名から座標への変換、全体の入口	受付係・司会
ProgramB.ino	経路生成、ライン追従、交差点検出、旋回、走行、終了処理	地図係・運転手
ProgramTypes.h	座標 vec、方角 DIR、動作段階 MODE の共通定義	共通の辞書

Arduinoでは、同じスケッチフォルダにある複数の .ino ファイルは一緒にビルドされます。そのため ProgramA から ProgramB の関数を呼びます。`extern` は「実体は別の場所にあるので、ここでは名前だけ知らせる」という宣言です。

2. コードを読むための最小限の言葉

言葉	やさしい意味	このコードの例
変数	途中で変わる値を入れる箱	cur, mode, targetCount
定数	実行中に変えない設定値	TURN_SPEED, GAP_TIMEOUT_MS
型	箱に入る値の種類	int=■■■■float=■■■■bool=■■■■char=1■■
関数	名前を付けたひとまとまりの処理	makeRoute(), lineTrace()
引数	関数へ渡す材料	addRouteCommand('f') ■ 'f'
戻り値	関数から返る結果	makeRoute() ■ true / false
配列	同じ型の箱を横に並べたもの	targetLabels[], com[]
文字列	char配列に文字を並べ、末尾を \0 で示すもの	"159b", "ffrfl"
条件分岐	条件により処理を選ぶ	if, else, switch
繰り返し	同じ処理を続ける	while, for
構造体	関連する値を1セットにまとめる	vec ■ x ■ y ■■■■
列挙型	選択肢にわかりやすい名前を付ける	DIR, MODE

短いコードの読み方

```
int error = reflectances.value(3) - reflectances.value(4);
int correction = error / 5;
```

1行目は3番センサー値から4番センサー値を引き、左右のずれを`error`に保存します。2行目は補正が急すぎないように5で割り、`correction`に保存します。セミコロン` `は1つの命令の終わりです。

大切な見方

コードは英単語を全部暗記する必要はありません。まず「どの箱を読み、どの箱を書き換え、どの関数を呼ぶか」を矢印で追うと理解しやすくなります。

3. このプログラムのデータ模型

座標 vec

```
struct vec
{
    int x;
    int y;
};
```

‘vec’ は1地点を表す自作の型です。‘vec cur = {0, 0};’ なら現在位置は (0,0)。`destination.x` のようにドットで中の値へアクセスします。

方角 DIR

名前	内部の整数	意味	座標の変化
NORTH	0	北	y が +1
EAST	1	東	x が +1
SOUTH	2	南	y が -1
WEST	3	西	x が -1

0→1→2→3の順が時計回りなので、`(dir + 1) % 4` で右を向けます。左は ‘+3’ (-1 と同じ) です。‘% 4’ は0〜3の範囲に巻き戻します。

動作段階 MODE

モード	意味	次へ進むきっかけ
COMMAND_INPUT_MODE	行き先を入力中	‘\n’ を入力
ROUTE_GENERATION_MODE	経路を作る段階	runOperation() 開始
RUN_MODE	校正後に走行する段階	開始ボタン
FINISHED_MODE	処理が終わった段階	再起動までそのまま

4. 起動から終了まで

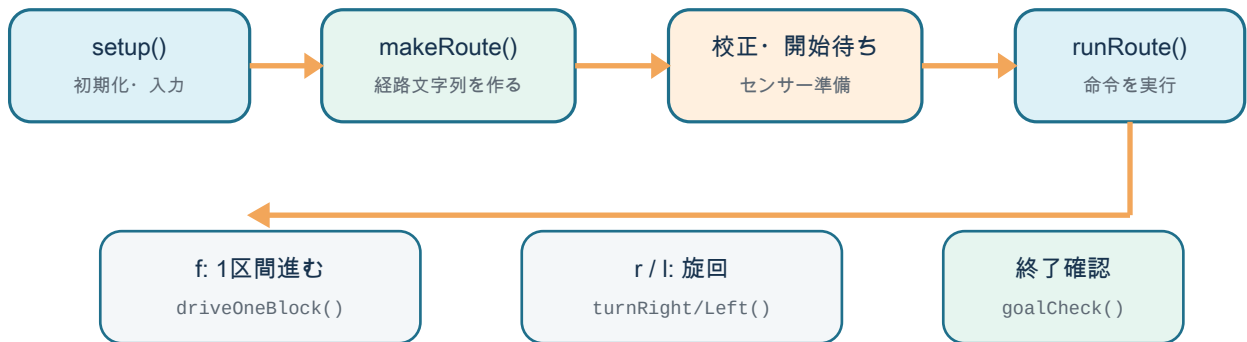


図2 最上位の処理フロー。runRoute() の中で f/r/l を1文字ずつ解釈します。

順番	起きること	主な関数 / 変数
1	通信、IMU、反射センサーを初期化する	<code>setup()</code> , <code>Serial</code> , <code>imu</code> , <code>reflectances</code>
2	行き先を1文字ずつ受け取る。`.` で確定	<code>getCommand()</code> , <code>targetLabels</code> , <code>targetCount</code>
3	地点を座標へ変換し、f/r/l の文字列を作る	<code>makeRoute()</code> , <code>labelToVec()</code> , <code>com</code>
4	ボタンを待ち、地磁気センサーを校正する	<code>calibrateCompass()</code>
5	ジャイロ用設定に替え、開始ボタンを待つ	<code>configureForTurnSensing()</code>
6	現在向きをコース上の北として記録する	<code>recordBaseHeading()</code> , <code>baseAngle</code>
7	com の先頭から旋回・前進を実行する	<code>runRoute()</code> , <code>indexCmd</code>
8	目的地一致を確認し、停止・ブザー・LED点滅	<code>goalCheck()</code> , <code>finishAction()</code>

setup() と loop()

Arduinoは電源投入後に `setup()` を1回だけ実行し、その後 `loop()` を繰り返します。このコードは最初の `loop()` で運行を最後まで行い、`mode` を `FINISHED_MODE` にするため、次の `loop()` では何もしません。

5. 変数・定数の完全一覧

グローバル変数は複数の関数から共有される箱です。設定値を調整すると挙動が変わるものと、運行中の状態を覚えるものがあります。

名前	初期値	役割	所在
MAX_TARGET	20	入力できる目的地数の上限。targetLabels は終端用1文字を足して21要素。	ProgramA
mode	COMMAND_INPUT_MODE	現在の動作段階。setup/loop/runOperation が更新。	ProgramA
targetLabels	char[21]	入力された目的地名の文字列。例: 159b。末尾は \0。	ProgramA
targetCount	0	有効な目的地の個数。追加時に増え、クリア時に0。	ProgramA
MAX_ROUTE_COMMAND	160	生成経路 com の配列サイズ。実命令は最大159文字。	ProgramB
cur	{0,0}	ロボットの論理的な現在座標。1区間走るたび更新。	ProgramB
goal	{0,0}	最後の入力地点の座標。運行後の一致確認に使う。	ProgramB
dir	NORTH	現在向いている論理方角。旋回成功後に更新。	ProgramB
com	char[160]	生成した走行命令列。f=前進、r=右、l=左。	ProgramB
commandLength	0	com に入っている命令数。追加位置にも使う。	ProgramB
routeOverflow	false	com に入りきらなかったことを記録する旗。	ProgramB
threshold	300	反射センサーがラインを見たと判断する境界値。	ProgramB
baseSpeed	60	通常前進時の左右モーター基準速度。	ProgramB
intersectionCenterDelays	970	交差点検出後、中心付近まで進む時間(ms)。	ProgramB
KpGap	2.0	欠線区間でジャイロ角ずれを速度差に変える比例係数。	ProgramB
TURN_SPEED	120	その場旋回時のモーター速度。	ProgramB
TURN_TOLERANCE_DEG	1.0	旋回後の地磁気方位の警告基準(度)。停止条件ではない。	ProgramB
GYRO_TURN_ANGLE_DEG	90.0	左右1回の目標回転角。	ProgramB
GYRO_TURN_TOLERANCE_DEG	3.0	ジャイロ旋回を終了する許容誤差(度)。	ProgramB
TURN_SETTLE_MS	150	停止後の慣性を測る待ち時間(ms)。	ProgramB
TURN_TIMEOUT_MS	8000	旋回が終わらない場合の上限時間(ms)。	ProgramB
BLOCK_TIMEOUT_MS	15000	1区間で次の交差点が見つからない場合の上限(ms)。	ProgramB
GAP_TIMEOUT_MS	5000	欠線中にラインが再発見できない場合の上限(ms)。	ProgramB
indexCmd	0	現在実行中の com の添字。命令ごとに増える。	ProgramB
baseAngle	0.0	開始時の実方位。論理 NORTH の基準として記録。	ProgramB

5. 変数を役割別に見る

グループ	変数	読み方
入力	targetLabels / targetCount	どこへ、何個行くか
経路	com / commandLength / routeOverflow	どの命令を、何文字作ったか、あふれたか
位置	cur / goal / dir / baseAngle	今どこで、どこを目指し、どちらを向くか
走行調整	threshold / baseSpeed / KpGap	ライン判定、速さ、欠線補正の強さ
旋回調整	TURN_SPEED / GYRO_* / TURN_*	旋回角、許容差、速度、時間制限
進行	mode / indexCmd	全体の段階と経路の読取位置

よく現れるローカル変数

名前	現れる場所	役割
input	getCommand	今読んだ1文字
unused	getCommand	入力が地点に変換できるか試す一時的な vec
destination	makeRoute	今回向かう地点の座標
routePosition	makeRoute	経路作成上の仮想現在位置。実機の cur はまだ動かさない
planningDir	■■■■■	経路作成上の仮想方角。ポインタ経由で連続更新
dx / dy	createRouteOneSection	目的地までのx/y方向の差
xPriority / yPriority	createRouteOneSection	現在の向きと移動方向の相性。大きい軸を先にする
diff	addTurnCommands	現在方角から目標方角まで時計回りに何段階か
error / correction	lineTrace	左右ずれと、そのずれから計算した速度補正
startedAt	■■■■■■■	時間切れを測る開始時刻
targetHeading	rotateToDirection	地磁気で見た最終的な目標方位
gyroAngle / gyroError	rotateToDirection	相対回転角と残り角度
finalHeading / compassError	rotateToDirection	停止後の絶対方位と目標との差
yaw	gapDrive	欠線開始後の相対的な曲がり角
leftStartIntersection	driveOneBlock	出発交差点を抜けたかを示す旗
centerAndStop	runRoute / driveOneBlock	交差点中心まで進んで停止するか
approachingStart	runRoute	最初のfがコース外→0の接近命令か
success / turned	runRoute	全体または旋回が成功したか

6. 関数の完全一覧 - ProgramA.ino

関数名の括弧内が引数、戻り値欄が関数から返る種類です。`void` は戻り値なしです。

関数	戻り値	役割	主な呼び出し元
labelToVec(char label, vec *p)	bool	地点文字を座標へ変換し p の中身を書き換える。対応外なら false。	getCommand, makeRoute
normalizeLabel(char label)	char	大文字A/Bを小文字a/bへそろえる。その他はそのまま返す。	getCommand
clearTargets()	void	目的地数を0にし、文字列先頭を \0 にして空にする。	getCommand
printInputPrompt()	void	入力方法と例をシリアルモニタへ表示する。	getCommand
getCommand()	void	有効な地点文字を蓄積し、*で全消去、.で入力確定する。	setup
setup()	void	通信とセンサーを初期化し、目的地入力を行う。起動時に1回。	Arduino■■■
loop()	void	経路生成モードなら runOperation を1回実行して終了モードにする。	Arduino■■■

ポインタ *p が必要な理由

`labelToVec('5', &destination;)` では、`&destination;` が destination の住所です。関数はその住所を `p` として受け取り、`p->x` と `p->y` を直接書き換えます。戻り値 `bool` は「変換できたか」の連絡に使えるため、座標そのものと成功可否を同時に返せます。

```
vec destination;
labelToVec('5', &destination);
// destination.x ■ 1■destination.y ■ 1 ■■■
```

6. 関数の完全一覧 - 経路生成

関数	戻り値	役割
<code>addRouteCommand(char command)</code>	<code>bool</code>	com末尾へ1文字追加。容量不足なら旗を立て <code>false</code> 。
<code>directionX(DIR direction)</code>	<code>int</code>	方角のx成分を <code>+1 / -1 / 0</code> で返す。
<code>directionY(DIR direction)</code>	<code>int</code>	方角のy成分を <code>+1 / -1 / 0</code> で返す。
<code>addTurnCommands(DIR target, DIR *planningDir)</code>	<code>void</code>	目標方角までの最少旋回を追加し、仮想方角を更新。
<code>addMoves(DIR target, int count, DIR *planningDir)</code>	<code>void</code>	必要なら旋回し、指定回数の <code>f</code> を追加。
<code>addXMoves(int dx, DIR *planningDir)</code>	<code>void</code>	<code>dx</code> の正負から東/西移動を追加。
<code>addYMoves(int dy, DIR *planningDir)</code>	<code>void</code>	<code>dy</code> の正負から北/南移動を追加。
<code>createRouteOneSection(vec start, vec destination, DIR *planningDir)</code>	<code>void</code>	2地点間の最短経路を追加。今の向きに近い軸を優先。
<code>makeRoute()</code>	<code>bool</code>	全入力地点をたどる <code>com</code> を作り、 <code>goal</code> を確定。

呼び出しの階層

```
makeRoute
  ■■ createRouteOneSection
    ■■ directionX / directionY
    ■■ addXMoves ■■
    ■■ addYMoves ■■■ addMoves
      ■■ addTurnCommands
      ■■ addRouteCommand
```

上の階層ほど「大きな目的」を表し、下へ行くほど小さな作業に分かれます。これは、長い処理を理解しやすく保つ典型的な設計です。

6. 関数の完全一覧 - センサー・位置・旋回

関数	戻り値	役割
lineTrace()	void	3番と4番の差から左右モーター速度を補正。
lineLost()	bool	1〜6番すべてがthreshold以下なら true。
detectIntersection()	bool	2番または5番がthreshold超なら交差点と判断。
updatePosition()	void	dirに応じてcurを1マス進める。
goalCheck()	bool	curとgoalのx/yが両方同じか返す。
normalizeHeading(float heading)	float	角度を0以上360未満に収める。
getHeading()	float	地磁気から平均方位を得て正規化。
recordBaseHeading()	void	開始時方位をbaseAngleへ保存し、論理方角をNORTHにする。
headingForDirection(DIR target)	float	論理方角を実際の目標方位へ変換。
shortestHeadingError(float target, float current)	float	-180〜180度の最短角度差を返す。
rotateToDirection(DIR target, float gyroTargetAngle)	bool	ジャイロで目標角まで旋回し、地磁気で最終確認。
turnRight()	bool	右隣の方角へ-90度旋回。
turnLeft()	bool	左隣の方角へ+90度旋回。

6. 関数の完全一覧 - 走行と統括

関数	戻り値	役割
gapDrive()	bool	ラインがない間、ジャイロで直進姿勢を保ち、再発見を待つ。
driveOneBlock(bool centerAndStop)	bool	出発交差点を抜け、次の交差点までライン追従。必要なら中心停止。
finishAction(bool success)	void	モーター停止、ブザー、結果表示、LED10回点滅。
runRoute()	void	comを順に解釈し、旋回と1区間走行を実行。最後に座標確認。
calibrateCompass()	void	地磁気の最大/最小値を測る校正を実行し表示。
runOperation()	void	経路生成→校正→開始待ち→基準方位記録→走行を統括。

外部ライブラリが用意するもの

reflectances (反射センサー)、motors (左右モーター)、imu (地磁気・ジャイロ)、button、buzzer、led は ZumoShieldN ライブラリ側の機能です。このプロジェクト内には定義が見えませんが、include により利用できます。

F("...") の意味

`Serial.println(F("Start running.));` の `F()` は固定文字列をArduinoのRAMではなくプログラムメモリ側に置くための仕組みです。表示内容は普通の文字列と同じですが、限られたRAMを節約できます。

7. 入力が経路になるまで

例: 159b. を入力

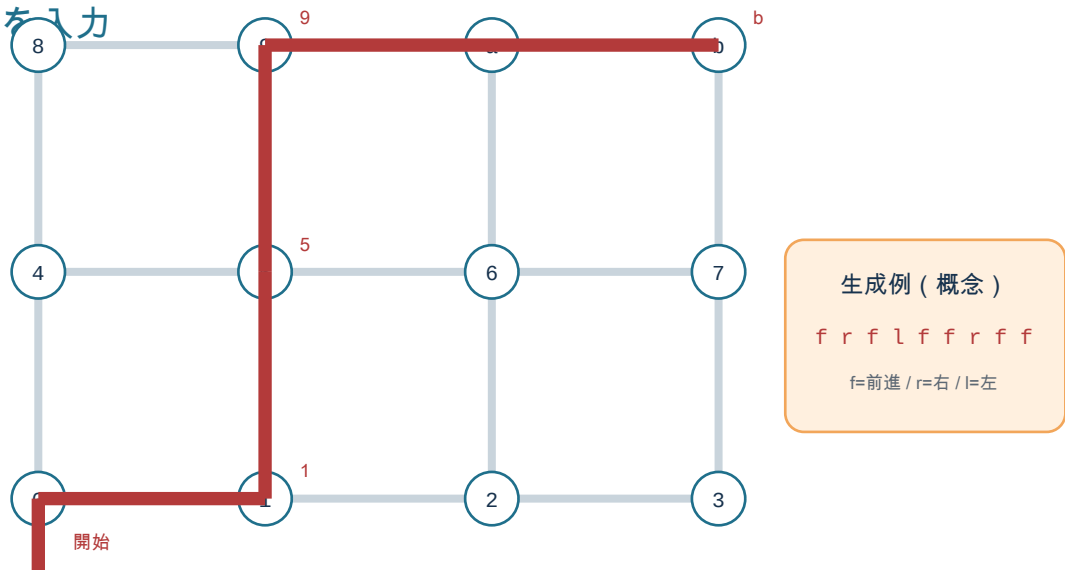


図3 入力地点を結ぶ経路のイメージ。実際の文字列は開始方角と軸の優先規則で決まります。

入力文字	getCommand() の動作
■■ / ■■	無視する
*	clearTargets() で全消去
.	1地点以上あれば入力終了。0件ならエラー
0■9, a, b, A, B	座標に変換できるか確認し、A/Bは小文字化して追加
■■■	Unsupported command と表示して無視
21■■■	全件を消去して Too many commands と表示

なぜ末尾に \0 を入れるのか

C/C++の文字列には長さ情報が付いていません。末尾の値0 (文字表記は `'\0'`) を「ここまで」とする約束です。追加するたび `targetLabels[targetCount] = '\0';`、経路でも `com[commandLength] = '\0';` として、表示や繰り返しが終端を見つけられるようにします。

7. 最短経路を作る考え方

2地点の差を `dx = destination.x - start.x`、`dy = destination.y - start.y`

で求めます。斜め移動はできないため、必要な前進回数は `|dx| + |dy|`。これがマンハッタン距離です。

```
start = (1, 0), destination = (3, 2)
dx = 2, dy = 2
■■■■■ = 2 + 2 = 4 ■
```

x方向とy方向のどちらを先に進んでも前進回数は同じです。そこで `xPriority` と `yPriority` を比較し、現在の向きにそのまま進みやすい軸を先にします。これにより最短距離を保ちつつ旋回回数を減らします。

方角差から旋回文字へ

diff	関係	追加命令
0	同じ向き	■■
1	右へ90度	r
2	反対向き	rr
3	左へ90度	l

```
`diff = (target - *planningDir + 4) % 4`
```

は、列挙値の循環を利用した式です。+4で負数を避け、%4で0〜3へ収めます。

実際の位置と計画上の位置を分ける

経路作成中はロボットがまだ動いていません。そのため cur / dir を変えず、routePosition / planningDir という一時的なコピーだけを進めます。この分離が、計画と実行の混同を防ぎます。

7. 経路配列の安全対策

`com` は160要素ですが、最後の1要素は `'\0'`

用です。よって命令を入れられるのは最大159文字。`addRouteCommand()` は追加前に長さをチェックします。

```
if (commandLength >= MAX_ROUTE_COMMAND - 1)
{
    routeOverflow = true;
    return false;
}
```

容量を超えた場合は `routeOverflow = true` にして、それ以上の追加を止めます。`makeRoute()`

は失敗を検出すると文字列を空に戻し、エラーを表示して `false`

を返します。これにより配列外へ書き込んでメモリを壊すことを防ぎます。

最初の f が特別な理由

ロボットの論理座標 `cur` は最初から(0,0)ですが、実機は交差点0の下にいます。そこで経路の先頭へ接近用の `f`

を必ず追加します。この前進を終えたときだけ `approachingStart` が true なので、`updatePosition()`

は呼ばれません。座標上は(0,0)のままです。

8. ラインを追う仕組み

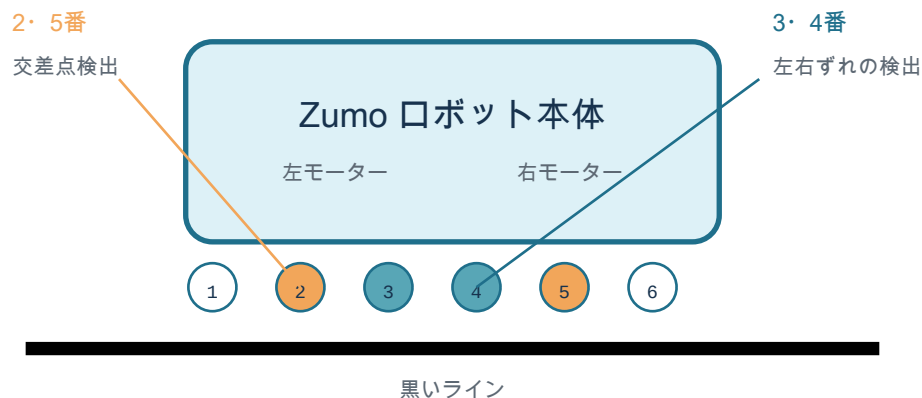


図4 反射センサーの使い分け。番号はコードが参照する1〜6番。

中央の3番と4番の値の差が左右ずれです。`error` が正なら補正值も正になり、左モーターを遅く、右を速くします。差が負なら逆です。こうしてライン中央へ戻る向きに曲がります。

```
error = sensor3 - sensor4
correction = error / 5
leftSpeed = baseSpeed - correction
rightSpeed = baseSpeed + correction
```

判定	使うセンサー	条件	意味
lineTrace	3, 4	差を計算	ライン中央へ速度補正
lineLost	1■6	全て threshold 以下	どのセンサーもラインを見ていない
detectIntersection	2 ■■■ 5	どちらかが threshold 超	横線が広がる交差点に来た

しきい値 threshold

センサー値が300を超えたらラインあり、という境界です。床、テープ、照明、センサー高さで適値が変わります。値が合わないと、交差点の見逃しや誤検出が起きます。

8. 1区間を走る driveOneBlock()

段階	判定と動作
開始	タイマーを記録し、まだ出発交差点内とする
時間切れ確認	15秒を超えたら停止して false
ライン消失	gapDrive() で姿勢を保って再発見を試す
出発交差点を抜ける前	交差点でない状態になるまで lineTrace() を続ける
次の交差点を検出	centerAndStop が true なら970ms直進して中心停止。成功 true
通常区間	lineTrace() を繰り返す

`leftStartIntersection` が重要です。旋回直後は出発地点の太い交差線をセンサーがまだ見ていることがあります。これを次の交差点と数えると、ほぼ移動せず成功扱いになります。いったん交差線が見えなくなったことを確認してから、次の検出を有効にしています。

いつ交差点中心で止まるか

次の命令も `f` なら、停止せず交差点を通過します。次が `r` / `l`
または終端なら、旋回または最終停止のため中心まで進みます。判定式は `com[indexCmd + 1] != 'f'`
です。文字列末尾には `\0` があるため、最後の次の文字も安全に読めます。

8. 欠線を渡る gapDrive()

コースの線が一時的に途切れていると、反射センサーだけでは進行方向を決められません。欠線に入った瞬間にジャイロ角を0へリセットし、その後の左右回転 `yaw` を0へ戻すようにモーター差を付けます。

```
yaw = imu.turnSensorAngleDegree();  
correction = KpGap * (0.0 - yaw);  
left = baseSpeed - correction;  
right = baseSpeed + correction;
```

これはP制御（比例制御）の最小形です。角度ずれが大きいほど強く戻し、小さければ弱く戻します。`KpGap` が大きすぎると左右に振動し、小さすぎると曲がったまま進みやすくなります。ラインを再発見すると `lineLost()` がfalseになり成功終了します。5秒以内に見つからなければ停止します。

ジャイロの役割

ジャイロは「最初から何度回ったか」という相対角度が得意です。地磁気は「世界に対してどちら向きか」という絶対方位が得意です。このコードは用途を分けています。

8. 90度旋回 rotateToDirection()

順番	処理	使う基準
1	目標DIRから絶対目標方位 targetHeading を計算	地磁気 + baseAngle
2	ジャイロ角を0へリセット	ジャイロ
3	目標が±90度になるまで左右逆回転	ジャイロ誤差
4	8秒超なら停止・失敗	millis()
5	停止後150ms、慣性回転も測定	ジャイロ
6	最終方位との差を求め、1度超なら警告	地磁気
7	論理方角 dir をtargetへ更新	プログラム状態

このコードのジャイロは反時計回りが正なので、左旋回は +90度、右旋回は -90度です。旋回中の停止条件はジャイロ誤差3度以内です。停止後の地磁気誤差1度は警告のみで、失敗にはしません。

角度の端をまたぐ計算

```
target = 2°, current = 358°  
■■■■■ = -356°  
■■■■■ = +4°
```

`shortestHeadingError()` は差が180度を超えたら360度を引き、-180度未満なら360度を足します。0度と359度が遠く離れているように誤解しないためです。

8. 経路を実行する runRoute()

`indexCmd` を0に戻し、`com[indexCmd]` が `\0` になるまで読みます。旋回文字が連続する場合 (180度の `rr` など) は内側のwhileで先にすべて実行し、その後に前進を1回実行します。

```
while (com[indexCmd] != '\0')
{
    while (com[indexCmd] == 'r' || com[indexCmd] == 'l') { ... }
    if (com[indexCmd] != 'f') { ■■■■; }
    driveOneBlock(centerAndStop);
    updatePosition();
    indexCmd++;
}
```

防御的な確認	起きたときの処理
旋回がfalse	success=false で運行ループ終了
f/r/l以外の生成命令	Invalid generated route として停止
1区間走行がfalse	success=false で終了
最終curがgoalと不一致	Final position does not match として失敗

最後は成功・失敗に関係なく `finishAction()`

を呼びます。そのため、どのエラー経路でもモーターを止める共通の出口があります。

9. エラー対策と時間制限

保護	条件	結果	目的
入力0件	. を入力したが targetCount==0	入力を続ける	目的地なし運行を防ぐ
入力過多	targetCount>=20	全入力を消去	配列外書き込みを防ぐ
経路過長	159命令を超える	経路を空にして中止	com破壊を防ぐ
巡回タイムアウト	8秒超	モーター停止・ false	回り続けを防ぐ
区間タイムアウト	15秒超	モーター停止・ false	交差点探索の暴走を防ぐ
欠線タイムアウト	5秒超	モーター停止・ false	ラインなし直進を防ぐ
不正経路文字	f/r/l以外	失敗終了	内部異常を検出
目的地不一致	cur != goal	失敗終了	論理位置のずれを検出

`millis()` はArduino起動後の経過ミリ秒を返します。開始時刻を保存し、`millis() - startedAt` と差を取る方法は、値の周回にも比較的強い定番パターンです。

安全上の注意

ソフトウェアの時間制限があっても、実機テストではロボットをすぐ止められる状態を保ち、車輪を浮かせた低速確認から始めてください。

9. 調整する値と症状

値	上げると	下げると	典型的な症状
threshold	ラインあり判定が厳しく/値の仕様次第	ラインあり判定が広がる	交差点誤検出・見逃し。実測値を確認
baseSpeed	速くなるが追従が難しい	安定しやすいが遅い	カーブで外れるなら下げる
intersectionCenterDelayMs	交差点の奥で止まる	手前で止まる	旋回中心がずれるとき調整
KpGap	欠線で強く向きを戻す	補正が穏やか	蛇行なら下げ、曲がるなら上げる
TURN_SPEED	旋回が速い	旋回が遅い	オーバーシュートなら下げる
GYRO_TURN_TOLERANCE_DEG	早めに旋回終了	より厳密に回る	小さすぎると収束しにくい
TURN_SETTLE_MS	惰性を長く観測	次処理へ早く進む	停止後の角度表示が不安定なら上げる

調整は一度に1項目だけ変え、変更前の値と結果を記録します。速度を変えると交差点中心までの時間も変わるため、`baseSpeed` と `intersectionCenterDelayMs` は関連します。

10. 1本の入力を追いかける

場面	状態の例	何が起きるか
起動直後	mode=COMMAND_INPUT_MODE, cur=(0,0), dir=NORTH	機器初期化後、入力待ち
159b. 入力後	targetLabels="159b", targetCount=4	入力確定
経路生成後	com=■■■f■■■■■■■■, goal=(3,2)	各地点間を順に連結
校正後	baseAngle=■■■■■■, dir=NORTH	実方位と論理北を結び付ける
命令f	driveOneBlock()	次交差点まで進む
命令r/l	turnRight/turnLeft()	その交差点で90度旋回
前進完了	cur■■dir■■■■1■■■■■■	最初の接近fだけ更新しない
終了0	cur■■goal■■■■	停止、結果表示、LED点滅

初心者向けデバッグの見方

シリアルモニタには入力文字列、生成経路、最終目的地、開始方位、旋回角、地磁気誤差、エラーが表示されます。まず`Generated route`と`Final goal`が想定どおりかを確認し、その後に実機の動きを見ます。計画の問題と走行制御の問題を切り分けられます。

10. コードを読む練習問題

問1: `dir` が EAST のとき `turnRight()` 後はどちら向きですか。

答え: `(1+1)%4=2` なので SOUTH。

問2: `cur=(2,1)`, `dir=WEST` で `updatePosition()` を呼ぶと？

答え: xが1減り、`cur=(1,1)`。

問3: `com[indexCmd]='f'`, 次も `f` のとき `centerAndStop` は？

答え: `com[indexCmd+1] != 'f'` はfalse。交差点中心で停止せず通過。

問4: すべての反射センサー値が threshold 以下なら？

答え: `lineLost()` がtrueになり、`gapDrive()` を試す。

問5: 経路作成中に `cur` を動かさないのはなぜ？

答え: 実機はまだ走っていないため。代わりに `routePosition` を進める。

変更するときの読み順

やりたいこと	最初に読む場所	関連箇所
入力できる地点を増やす	labelToVec / MAX_TARGET	■■■■■■■■■■
速度を変える	baseSpeed / lineTrace	■■■■■■■■■■
経路の選び方を変える	createRouteOneSection	addXMoves / addYMoves
旋回精度を上げる	rotateToDirection	TURN_SPEED, ■■■, ■■
欠線対応を変える	gapDrive	KpGap, GAP_TIMEOUT_MS

用語集

用語	このプログラムでの意味
シリアルモニタ	PCとArduinoが文字でやり取りする画面
ライントレース	床のラインをセンサーで見て追いつける走行
反射センサー	床から返る光の強さでラインを判断するセンサー
IMU	地磁気センサーやジャイロなど、姿勢・向きを測る装置
地磁気 (コンパス)	絶対的な向きを測る。周囲の金属や磁石の影響を受ける
ジャイロ	短時間の回転量を測る。相対的な角度変化が得意
キャリブレーション	センサーの偏りや範囲を実環境に合わせて測る作業
マンハッタン距離	縦横移動だけの最短距離。 $ dx + dy $
P制御	誤差に比例した強さで修正する制御
タイムアウト	処理が長すぎると安全のため失敗終了する仕組み
ポインタ	データの置かれた場所 (住所) を表す値
ヌル終端	文字列末尾を <code>\0</code> で示すC/C++の約束
グローバル変数	複数の関数から共有できる変数
ローカル変数	関数やブロックの中だけで使う一時的な変数

最後に

このコードの中心は、地点列 → 座標列 → f/r/l命令列 → 実機動作

という段階的な変換です。わからなくなったら、今見ている値がこの4段階のどこにいるかを確認してください。